

Python: Kontrollstrukturen

Arbeitsauftrag mit je einem Beispiel pro Konzept

1. Kontrollstrukturen (Python-Syntax)

if / elif / else

Verzweigung: Je nach Bedingung wird genau ein Block ausgeführt.

```
punkte = 78
if punkte >= 90:
    note = 1
elif punkte >= 75:
    note = 2
else:
    note = 3
print(note) # 2
```

Schleifen (for und while)

for läuft über eine Folge, while läuft, solange die Bedingung wahr ist.

```
for i in range(3):
    print("for:", i)      # 0, 1, 2

zaehler = 0
while zaehler < 3:
    print("while:", zaehler)
    zaehler += 1
```

break

Beendet die Schleife sofort.

```
for zahl in [3, 8, 15, 4]:
    if zahl > 10:
        print("Erste Zahl > 10:", zahl)
        break
```

continue

Springt zum nächsten Schleifendurchlauf.

```
for zahl in range(1, 6):
    if zahl % 2 == 0:
        continue
    print(zahl)          # 1, 3, 5
```

pass

Platzhalter, der nichts tut. Nötig, wenn ein Block syntaktisch nicht leer sein darf.

```
def spaeter_implementieren():
    pass

for zahl in range(3):
```

```
if zahl == 1:
    pass          # hier soll später etwas passieren
print(zahl)
```

try / except / else / finally

Fehlerbehandlung: except fängt Fehler ab, else läuft nur ohne Fehler, finally läuft immer.

```
try:
    x = int(input("Zahl: "))
    ergebnis = 10 / x
except ValueError:
    print("Das war keine Zahl!")
except ZeroDivisionError:
    print("Durch 0 teilen geht nicht!")
else:
    print("Ergebnis:", ergebnis)
finally:
    print("Fertig")
```

2. Bedingte Ausdrücke (ternärer Operator)

Aufbau: wert_wenn_wahr if bedingung else wert_wenn_falsch

```
alter = 17
status = "volljährig" if alter >= 18 else "minderjährig"
print(status) # minderjährig
```

3. Openbook Rheinwerk (Kapitel 5, Kontrollstrukturen)

https://openbook.rheinwerk-verlag.de/python/05_001.html

Beispiel: for mit else. Der else-Block läuft nur, wenn die Schleife ohne break beendet wurde.

```
for zahl in [2, 4, 6]:
    if zahl % 2 != 0:
        print("Ungerade gefunden")
        break
else:
    print("Alle Zahlen sind gerade") # wird ausgegeben
```

4. switch-case in Python (Datacamp)

<https://www.datacamp.com/tutorial/python-switch-case>

Python hat kein klassisches switch. Es gibt zwei gängige Lösungen.

a) match-case (ab Python 3.10)

```
tag = 3
match tag:
    case 1:
        print("Montag")
    case 2:
        print("Dienstag")
    case 3:
        print("Mittwoch")
    case _:
        # default
```

```
print("Unbekannt")
```

b) Dictionary als switch

Funktioniert auch in älteren Python-Versionen.

```
tage = {1: "Montag", 2: "Dienstag", 3: "Mittwoch"}  
print(tage.get(3, "Unbekannt")) # Mittwoch
```